

Exploring Transfer Learning and Fine-tuning with FiftyOne's Model Zoo

Author: Antonio Rueda-Toicen

The “Bo or Not Bo” Challenge

For modern neural networks, we rarely train models from scratch. Instead, we use transfer learning, which involves taking a network pre-trained on a particular dataset and task and adapting it to another. This approach saves time, reduces data requirements, and often leads to better performance.

This tutorial provides a hands-on demonstration of transfer learning. We will adapt a VGG16 model from the FiftyOne Model Zoo, originally trained to classify 1000 object types from the massive ImageNet-1k dataset, into a custom binary classifier. This new model will be trained with just a few dozen examples on a very imbalanced dataset.

Our challenge, taken from [NVIDIA's “Fundamentals of Deep Learning” course](#), is to distinguish images of former President Barack Obama's dog, Bo, from other images. I have often used FiftyOne to enrich and expand my workshops, and I hope that you find it useful too!

Our guided tour

This blog post is a guided walkthrough of the hands-on experience. Please follow along using the [accompanying Google Colab notebook](#).



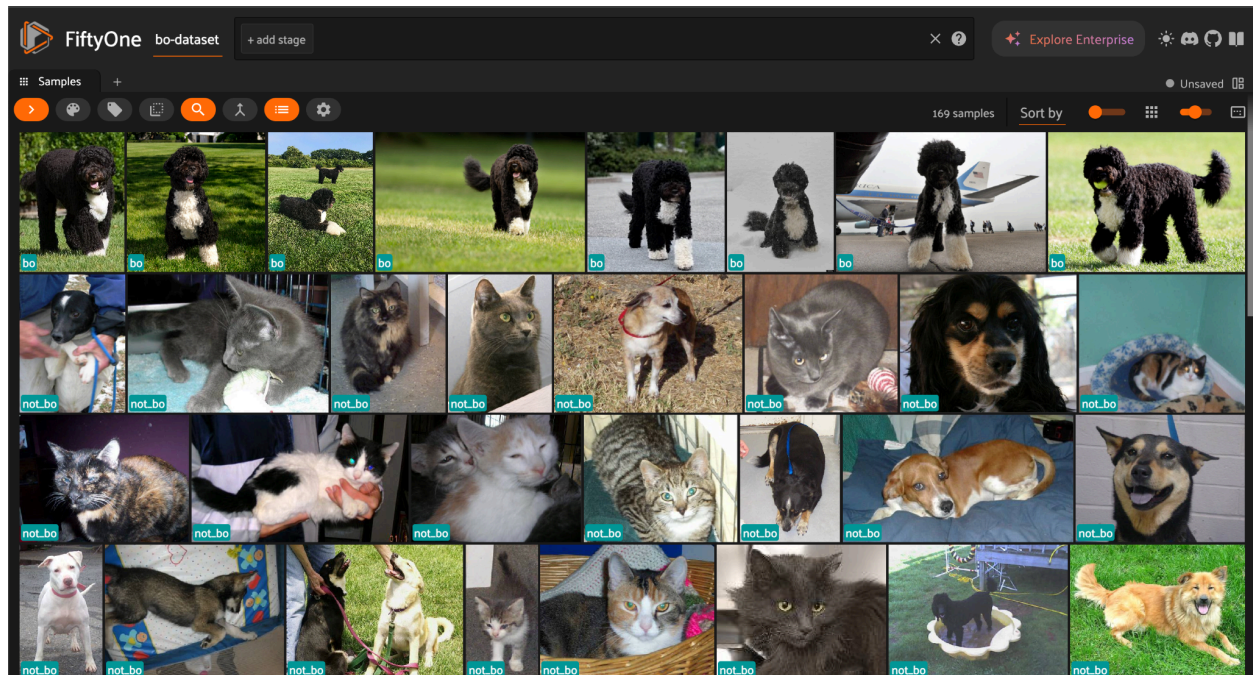
Bo in the Oval Office

The “Bo or Not Bo” dataset is an ideal case study because it mirrors real-world machine learning challenges, including:

- **Class imbalance:** An uneven number of “Bo” vs. “Not Bo” images.
- **Data quality issues:** Irrelevant or corrupted files mixed in with the dataset.
- **Data leakage:** The same images appear in both training and testing sets.

We will use the open-source tool **FiftyOne** to navigate these challenges and build a robust classification pipeline.

Meet Bo: The Classification Challenge



Bo Dataset Overview

The “Bo or Not Bo” dataset is a very small dataset of less than 200 images of Bo and other pets. This binary classification problem provides an ideal learning environment for understanding transfer learning concepts while working with real-world data challenges.

The dataset exhibits several characteristics common in practical machine learning scenarios:

- **Data scarcity:** we seldom have the opportunity of collecting thousands of images before starting a project
- **Class imbalance:** The distribution between “Bo” and “Not Bo” images is uneven, this problem is complicated by the small size of the dataset
- **Data quality issues:** Some images contain artifacts (like overlaid text) or irrelevant content (having mostly background content)
- **Split contamination:** Potential data leakage between training, testing, and validation sets

These challenges make our dataset perfect for demonstrating how FiftyOne’s tools can identify and resolve common data issues while building robust classification pipelines.

What is Transfer Learning?

Transfer learning leverages knowledge gained from solving one task to improve performance on a related task. In computer vision, it's popular to use models pre-trained

on ImageNet-1K, which contains more than a million images spread across 1000 object categories. We often use this dataset to train custom Convolutional Neural Networks. These models learn hierarchical feature representations:

- **Lower layers:** Detect edges, textures, and basic patterns
- **Middle layers:** Recognize shapes, object parts, and spatial relationships
- **Higher layers:** Capture complex object-specific features

The ImageNet Foundation

ImageNet's scale and diversity make it an excellent foundation for transfer learning. Models trained on this dataset develop robust feature extractors that generalize well to new domains. VGG16, our chosen architecture, achieved breakthrough performance on ImageNet-1k and remains popular for transfer learning due to its straightforward design and strong feature extraction capabilities.

Why Transfer Learning Works

Transfer learning succeeds because visual features learned on one dataset often prove useful for other visual tasks. The convolutional filters that detect edges in ImageNet images work equally well for detecting edges in photographs of people. This reusability of features allows us to:

1. **Reduce training time:** start with learned features instead of random initialization
2. **Improve performance:** leverage sophisticated image features learned from massive datasets
3. **Handle small datasets:** with transfer learning we are often able to achieve good results even with limited training data

Setting Up FiftyOne and Loading the Dataset

FiftyOne provides powerful tools for dataset management, visualization, and model integration. In the [Google Colab notebook](#) we obtain the [dataset from Kaggle](#) by using kagglehub and connect to Google Drive's filesystem. You can also download the output of that notebook [from HuggingFace's hub](#).

For now, let's abstract from those details and say that we begin by creating a FiftyOne dataset from our image collection:

```
import fiftyone as fo
import fiftyone.zoo as foz

# Create FiftyOne dataset
dataset = fo.Dataset("bo_dataset")
dataset.add_dir(
    dataset_dir=data_path,
    dataset_type=fo.types.ImageClassificationDirectoryTree,
)
```



```
# Verify dataset creation
print(f"Dataset contains {len(dataset)} images")
print(f"Classes: {dataset.distinct('ground_truth.label')}")
```

This creates a structured dataset where FiftyOne can track metadata, predictions, and analysis results throughout our workflow. One valuable feature of FiftyOne is being able to immediately tag and filter train, testing, and validation splits. We will use this feature throughout our demonstration.

```
# Create views for each tag

train_view = bo_dataset.match_tags("train")

validation_view = bo_dataset.match_tags("validation")

test_view = bo_dataset.match_tags("test")


# Save the views, we can access them via the FiftyOne app
bo_dataset.save_view("train_view", train_view, overwrite=True)
bo_dataset.save_view("validation_view", validation_view, overwrite=True)
bo_dataset.save_view("test_view", test_view, overwrite=True)
```

Remember that we use the training set to update the parameters of the model, the validation set to find its optimal settings (aka “hyperparameters”), and the test set to provide a final and unbiased estimate of the model’s performance.

Applying the Pre-trained VGG16 Model

FiftyOne’s Model Zoo provides easy access to pre-trained models. We load VGG16 and apply it to our dataset for initial exploration:

```
# Load pre-trained VGG16 from FiftyOne Model Zoo
model = foz.load_zoo_model("vgg16-imagenet-torch")


# Apply model to dataset
dataset.apply_model(model, label_field="vgg16_predictions")
```

This generates ImageNet predictions for each image, allowing us to explore how the pre-trained model interprets our data before adaptation.

Visualizing Feature Representations

Understanding the internal representations learned by VGG16 provides insights into the changes that we need to make for transfer learning. We can inspect the output of the model by running:

```
model = foz.load_zoo_model("vgg16-imagenet-torch")

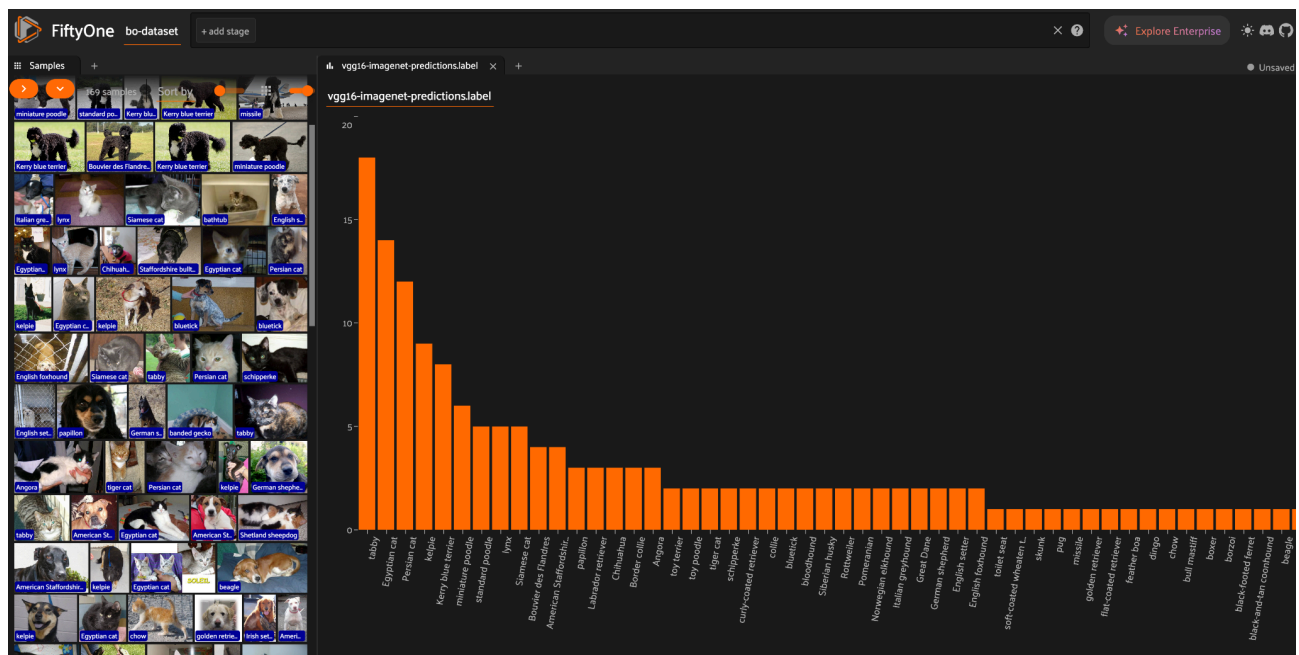
# We obtain the predictions of the original model's architecture

bo_dataset.apply_model(model,
label_field="vgg16-imagenet-predictions")
```

We can then observe the class distribution of the samples in our dataset by going to + -> Histograms and selecting 'vgg16-image-predictions-label'. We can split the panel horizontally to see the data next to the distribution.

Why the Pre-trained Model Fails

We can verify that the breed of Bo, Portuguese Water Dog, is not among the classes present in the pretrained model's output.



Bo is recognized as different poodle or terrier breeds in a variety of photos. The 1000 dimensional output vector that we produce in the final layer of VGG16 is specific to the objects and breeds of pets present in the Imagenet1k taxonomy.

Extracting Embeddings from the Pre-trained Model

Embeddings from the pre-trained model capture high-level visual features that we can visualize and explore. Similar images cluster together in the embedding space, revealing patterns in our dataset that inform our transfer learning approach.

Think of an embedding as a dense numerical coordinate for an image, much like latitude and longitude represent a city on a map. Having been trained on millions of images, the VGG16 model has learned to create coordinates where images with similar content (e.g., two different fluffy dogs) have very similar numerical values, while

dissimilar images (e.g., a dog and a car) have very different values. This is why we can use the mathematical distance between these coordinates to find visually similar images.

We extract embedding vectors from the penultimate fully connected layer of VGG16. Each of these embeddings is a 4096-dimensional vector that has not been made "task-specific". This means that these embeddings don't map to one of the 1000 ImageNet-1K classes. The final layer of the VGG16 network performs that mapping task and has a thousand units. FiftyOne allows us to extract both these 4096-dimensional embeddings and final prediction output (a thousand different scores, one for each class in the ImageNet-1K dataset, per each image fed to the network) from the same pretrained model.

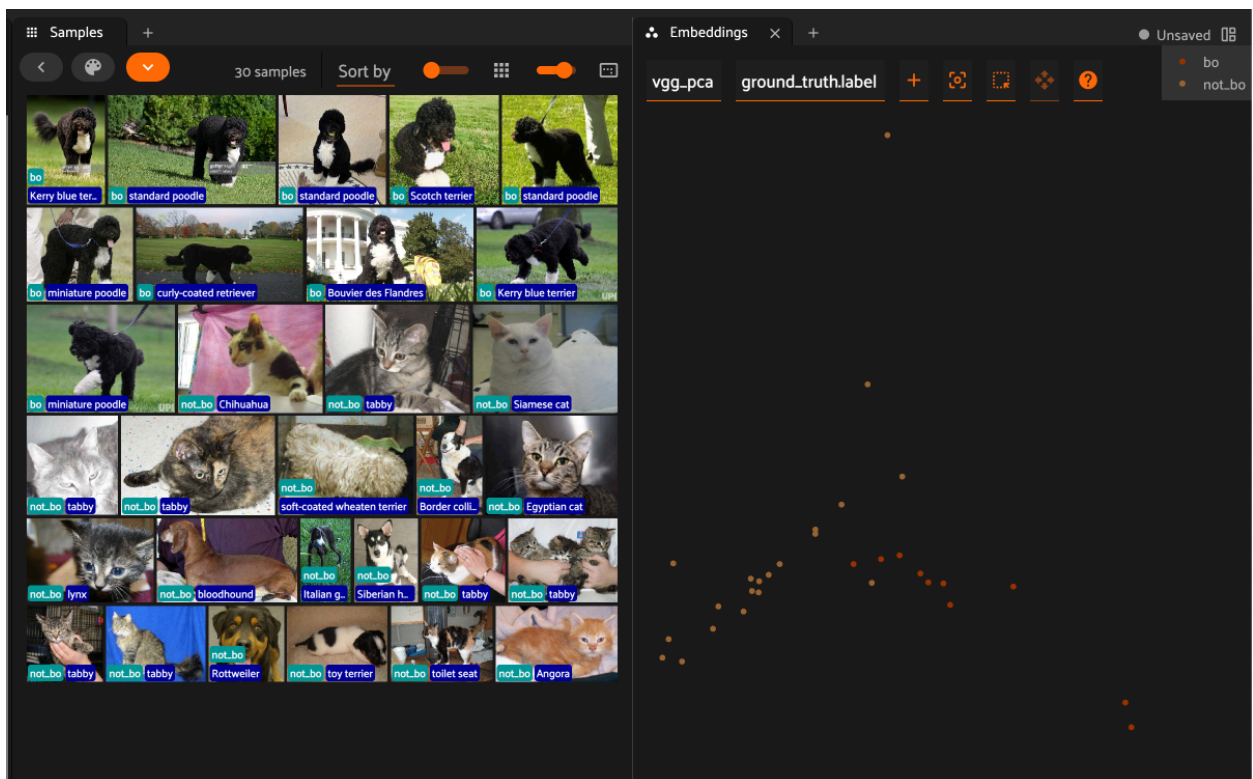
To get these rich feature representations, we instruct FiftyOne to process our images with the VGG16 model but to stop just before the final classification layer. Instead of getting one of 1000 class scores, we grab the output of the penultimate layer: a dense 4096-dimensional vector. This vector is the model's high-level summary of the image's content, perfect for tasks like similarity search.

```
# Compute embeddings from pre-trained features
dataset.compute_embeddings(model,
    embeddings_field="vgg16_embeddings")
```

We project these embeddings with a dimensionality reduction technique like principal component analysis (PCA) to observe them within the FiftyOne app.

```
visualization = fob.compute_visualization(bo_dataset,
                                          method='pca',
                                          brain_key='vgg_pca')
```

To visualize the embedding space within the FiftyOne app, we click the + symbol next to `Samples`, then select `Embeddings` and then the key `vgg_pca`. It's useful to split the panel horizontally so that we can see the original images next to the embeddings and select portions of the image-embedding space with the lasso tool.



Embedding Visualization

Inside the Fiftyone Embeddings panel, we can notice that the space of projected Bo embeddings looks quite distinct from the rest of the data.

Embeddings also allow us to create a similarity index for each image, which helps us uncover hidden patterns in individual data points, such as the presence of near or exact duplicates.

```
# Create similarity index for exploration
dataset.create_similarity_index("vgg16_embeddings")

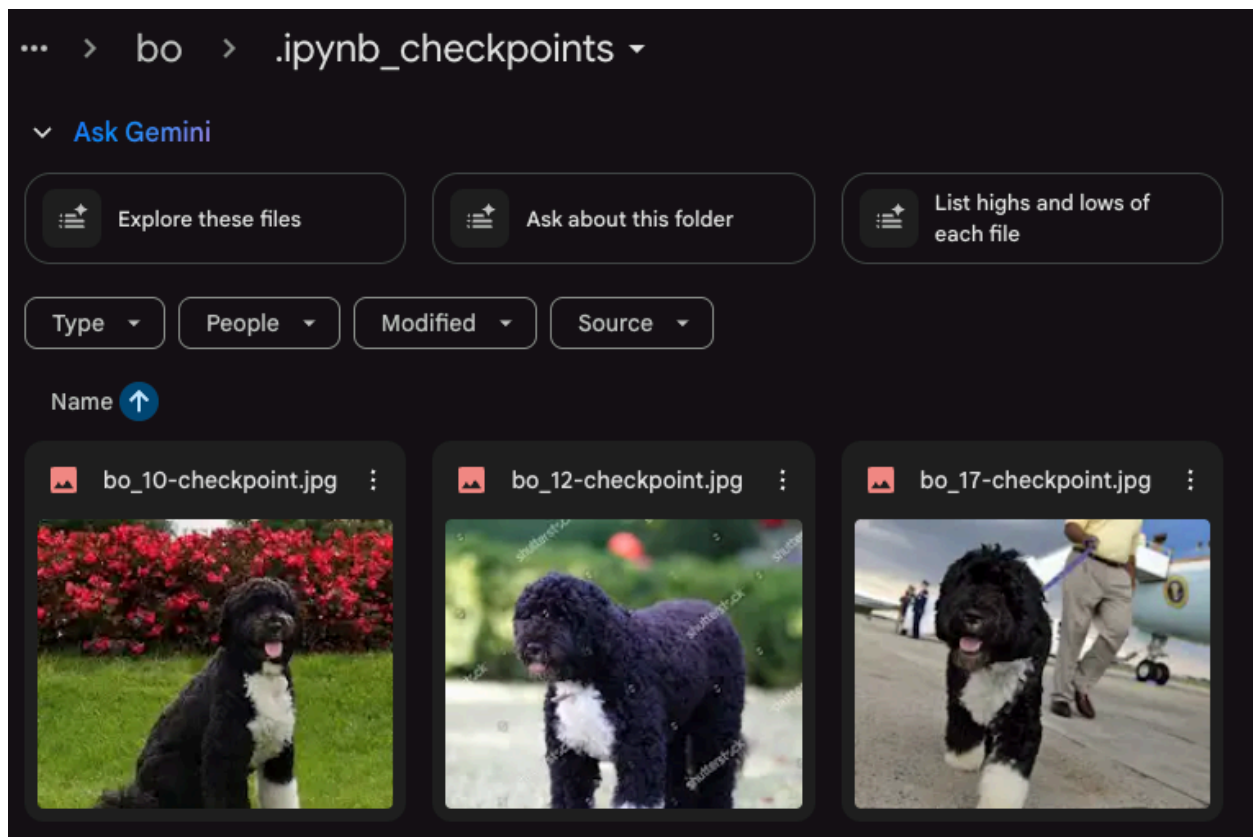
similar_view = bo_dataset.sort_by_similarity(
    a_sample.id,
    brain_key="vgg16_imagenet_cosine_similarity_index",
    k=5, # Return top k similar images
    reverse=False # Most similar first
)

# We save the view so that we can retrieve it later
bo_dataset.save_view('most_similar_to_sample', similar_view)
```



Data Quality: Removing Corrupted Checkpoint Files

Our similarity analysis exposes images in our dataset containing “checkpoint” duplications from the download process. This is a very common and annoying issue when working with files downloaded while working on a Jupyter or Google Colab notebook. Below you can see some of the ones that appear on my own Google Drive.



Checkpoint files in Google Drive

We can quickly remove these using FiftyOne's SDK:

```
# Find checkpoint files
checkpoint_view = dataset.match(F("filepath").contains("checkpoint"))
print(f"Found {len(checkpoint_view)} checkpoint files")

# Remove checkpoint files
dataset.delete_samples(checkpoint_view)
```

Identifying Data Leakage

Data leakage occurs when identical or near-identical images appear in both training and validation sets. This leads to overly optimistic performance estimates. Data leakage is like studying with tomorrow's exam questions. Our practice scores look great, but we will fail on truly new material. In production ML systems, this translates to models that seem perfect in testing but fail catastrophically on real users' data. FiftyOne can detect potential leaks using embedding similarity:

```
# Define your splits via tags
split_tags = ["train", "test", "validation"]

# Compute leaky splits
```

```

index = fob.compute_leaky_splits(bo_dataset,
                                splits=split_tags,

embeddings="vgg16-imagenet-embeddings")

leaks = index.leaks_view()

```



Data Leaks Detection

Data Leaks Detection

This analysis reveals which images appear in multiple splits, allowing us to create clean train/validation boundaries. This analysis reveals which images appear in multiple splits. As shown in the notebook, we can then programmatically remove these duplicates to ensure our sets are clean:

```

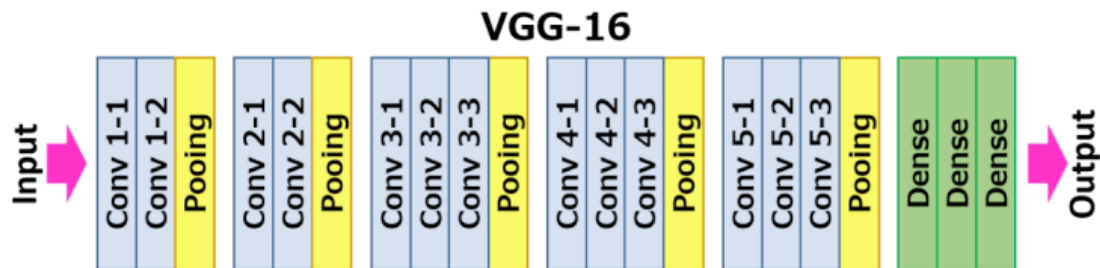
train_set = index.split_views["train"]
train_set_no_leaks = index.no_leaks_view(train_set)

```

It's important to understand that removing these duplicates will **decrease** the reported precision and recall of our model during evaluation of the test set. However, if we were to proceed without addressing this leakage, our model's validation accuracy would be artificially inflated. It would seem to perform brilliantly because it's being tested on images it has already seen during training. This is a critical failure in machine learning practice, as the model would likely perform much worse on truly new and unseen data. By using FiftyOne to find and remove these leaks, we ensure our final performance metrics are honest and reliable.

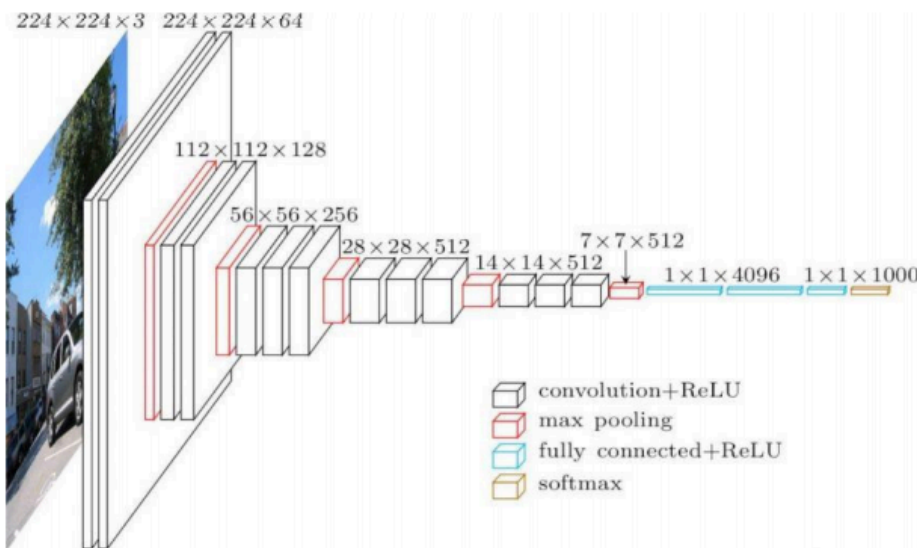
Adapting VGG16 for Binary Classification

The pre-trained VGG16 model outputs predictions for 1000 ImageNet classes. For our binary task, we must modify the architecture. While more modern architectures exist, VGG16's straightforward, sequential design makes it a good didactic model. Its distinct separation between the convolutional 'feature extractor' base and the final 'classifier' head makes the process of adaptation for transfer learning clear to visualize and implement.



The Architecture

The architecture depicted below is VGG16.



VGG16 Architecture

Why VGG16 and not ResNet, EfficientNet, ViT, or DINOv3?

VGG16 came out in 2014, which makes it ancient by deep learning standards. It's not the fastest horse in the race, nor the strongest, but it is the most straightforward. Think of it as the “hello world” of modern convolutional networks: easy to train, easy to tweak, and easy to explain. That's why it shines as a teaching tool for transfer learning. The tricks you learn here (freezing layers, swapping heads, handling imbalance) carry over almost unchanged to ResNets, EfficientNets, or even vision transformers. VGG16 just lets us see the moving parts without all the extra gears.

Model Architecture Adaptation

We replace the thousand output units of the original VGG16 classifier with a single unit. This frames the problem as binary classification. “Bo” is the positive class (1) and “Not Bo” is the negative class (0).

```

import torch.nn as nn

# Load pre-trained VGG16
vgg16 = torchvision.models.vgg16(pretrained=True)

# Freeze convolutional layers
for param in vgg16.features.parameters():
    param.requires_grad = False

# Replace classifier layers
vgg16.classifier = nn.Sequential(

    # First dense layer: compress 25088 features to 4096

    # This creates a bottleneck that forces the model to learn
    # the most important representations

    nn.Linear(25088, 4096),

    # ReLU activation: allows non-linear decision boundaries # Critical
    # for learning complex patterns
    nn.ReLU(),

    # Dropout: randomly zero 50% of neurons during training

    # Prevents overfitting on our small dataset
    nn.Dropout(0.5),

    # Final layer: 4096 -> 1 for binary classification

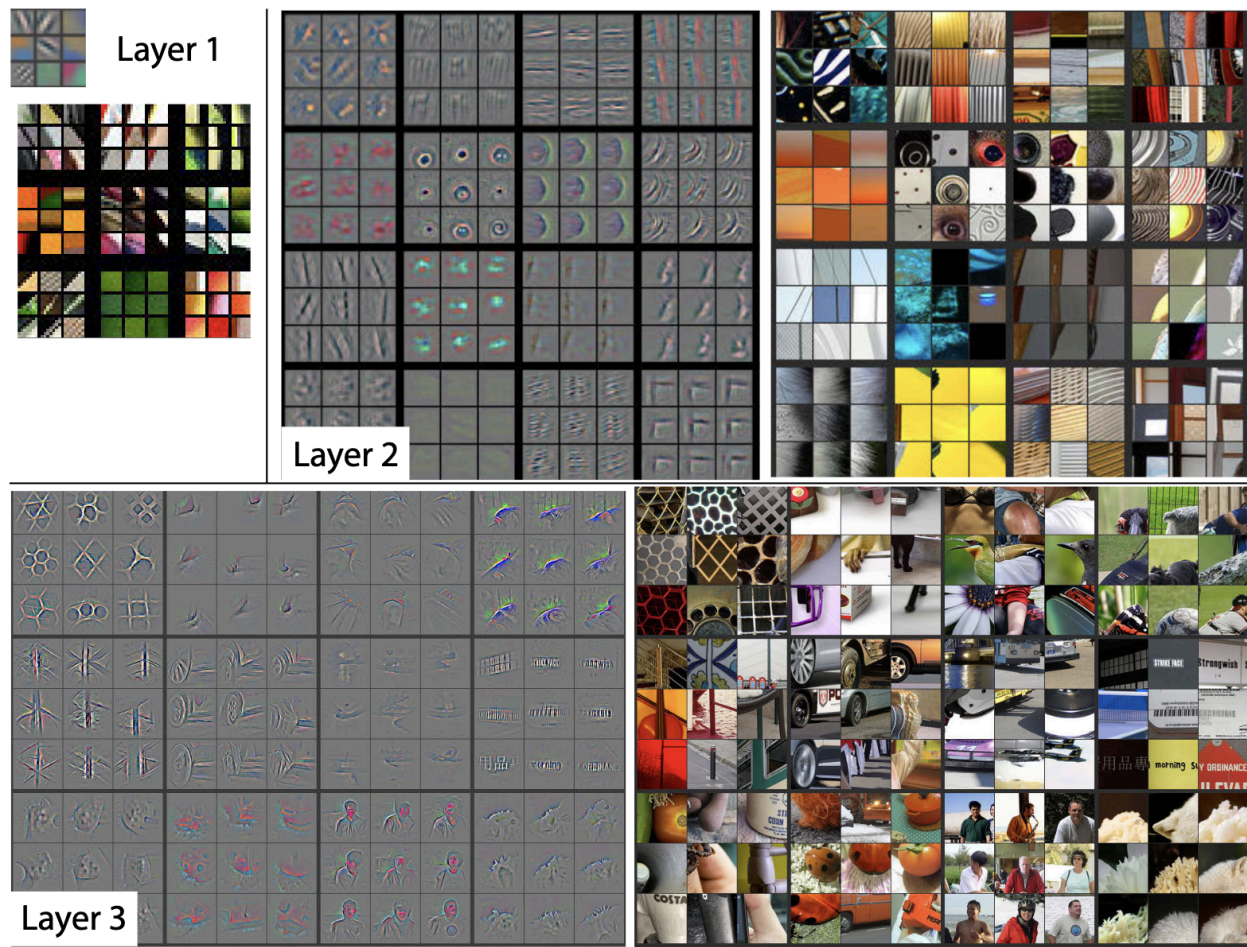
    # Output will be fed to sigmoid for probability
    nn.Linear(4096, 1),
)

```

`nn.Sequential` is simply a container that stacks layers in order. We have added two fully connected layers (aka `Linear`), a ReLU non-linear activation, and a Dropout layer to prevent overfitting. The 25088 features in the first fully connected layer are derived from the output of VGG16's final convolutional layer, which has a shape of (512, 7, 7). When we flatten this, we get $512 * 7 * 7 = 25088$. This is an important step to connect the convolutional feature extractor to our new dense classifier.

Why Freeze Base Layers?

The initial layers of a CNN learn to recognize very general features (edges, colors, textures) that are useful for almost any image-related task. By freezing them, we not only save computational resources but also prevent the model from "forgetting" these valuable, general-purpose features that it learned from ImageNet-1K while it learns the more specific features of the new dataset. The image below, from "[Visualizing and Understanding Convolutional Neural Networks](#)" illustrates this.



Freezing the convolutional base preserves the learned feature extractors while allowing the new classifier layers to adapt to our specific binary classification task. This approach:

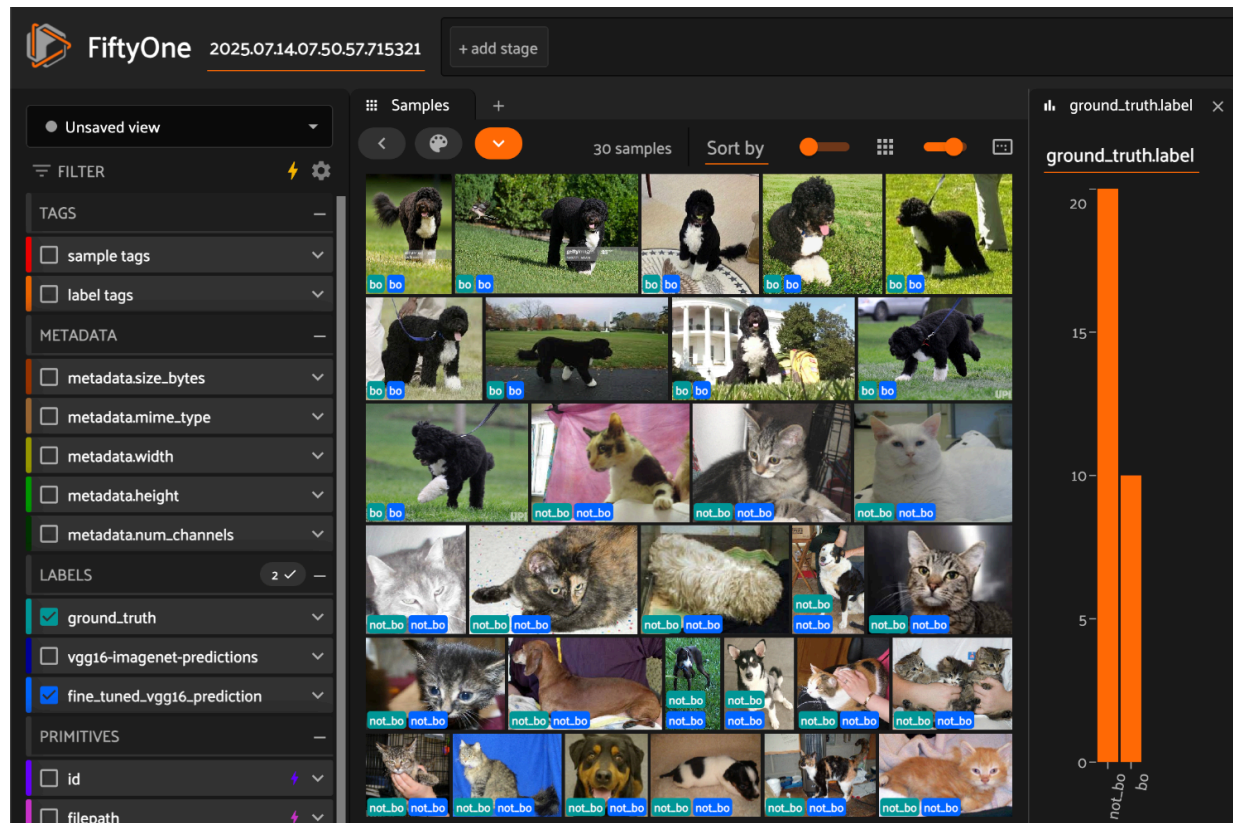
1. **Prevents overfitting:** Reduces trainable parameters when working with small datasets
2. **Preserves learned features:** Maintains ImageNet-learned representations
3. **Focuses learning:** Concentrates parameter updates on task-specific layers

We could compare the frozen convolutional layers to the human visual cortex, which has learned to recognize basic shapes, edges, and textures over a lifetime. When learning to identify a new type of object (like an exotic mineral or plant that we have never seen before) we don't relearn how to see edges, we just learn how the new object is composed of those fundamental features.

The Critical Role of Weighted Binary Cross-Entropy Loss

One of the most common challenges in real-world datasets is class imbalance, and the 'Bo or Not Bo' set is no exception.

Using FiftyOne, we can immediately visualize this problem by creating a histogram of our training data labels.



Class Distribution

We can observe this through the FiftyOne app by navigating to '+' -> Histograms and selecting `ground_truth.label` from the dropdown menu. We can produce a visualization of the class distribution on the training set. If we click on 'Split Horizontally', we see the images alongside the histogram.

Understanding the Problem

Imagine you're a teacher grading exams where 90% are easy questions and 10% are hard ones. If you weigh them equally, students will focus only on easy questions. Likewise our model will ignore the rare 'Bo' class unless we make those mistakes more expensive.

Class imbalance can bias model training towards the majority class. We can address this using a weighted loss function that penalizes misclassification of minority class

examples more heavily. In our code we use the weighted binary cross entropy loss to perform experiments on the effect of considering the class imbalance during training.

We calculate the weight that we assign to the positive class by computing the ratio between the two classes and call it `pos_weight`. Remember that `train_view` contains all samples from the training set. We can leverage the tight integration between PyTorch and FiftyOne to compute this with a few lines of code.

```
# Calculate class weights based on inverse frequency

pos_samples = len(train_view.match(F("ground_truth.label") ==
"Bo"))

neg_samples = len(train_view.match(F("ground_truth.label") ==
"Not Bo"))

pos_weight = torch.tensor([neg_samples / pos_samples])

# Define the weighted loss function

criterion =
nn.BCEWithLogitsLoss(pos_weight=pos_weight.to(device))
```

The `pos_weight` parameter scales the loss for positive examples. If we have 3 times more “Not Bo” images than “Bo” images, `pos_weight` will be 3.0, making each “Bo” misclassification three times more costly than a “Not Bo” misclassification.

Experiment with this: [the notebook](#) provides a direct comparison between weighted and unweighted approaches. I encourage you to run the notebook first *without* passing `pos_weight` to the loss function. Notice the errors that will appear on the final evaluation of the model. Then try running the notebook again including the weighted version of the loss.

As we saw in our FiftyOne histogram, we have far more 'Not Bo' images than 'Bo' images. Without intervention, a 'lazy' model could achieve decent accuracy simply by guessing 'Not Bo' most of the time. The `pos_weight` parameter acts as a counterbalance. By setting `pos_weight` to `[neg_samples / pos_samples]`, we are telling the loss function: 'A mistake on a single 'Bo' image is now just as costly as multiple mistakes on 'Not Bo' images.' This forces the model to pay close attention to the rare, positive class.

Why This Matters in Practice

Class imbalance appears in many real-world scenarios:

- **Medical diagnosis:** Rare diseases vs. common conditions
- **Fraud detection:** Fraudulent vs. legitimate transactions
- **Quality control:** Defective vs. normal products

Understanding weighted loss functions prepares you for these practical challenges. The “Bo or Not Bo” dataset provides a controlled environment to experiment with these concepts and observe their effects firsthand.

Your Turn to Experiment

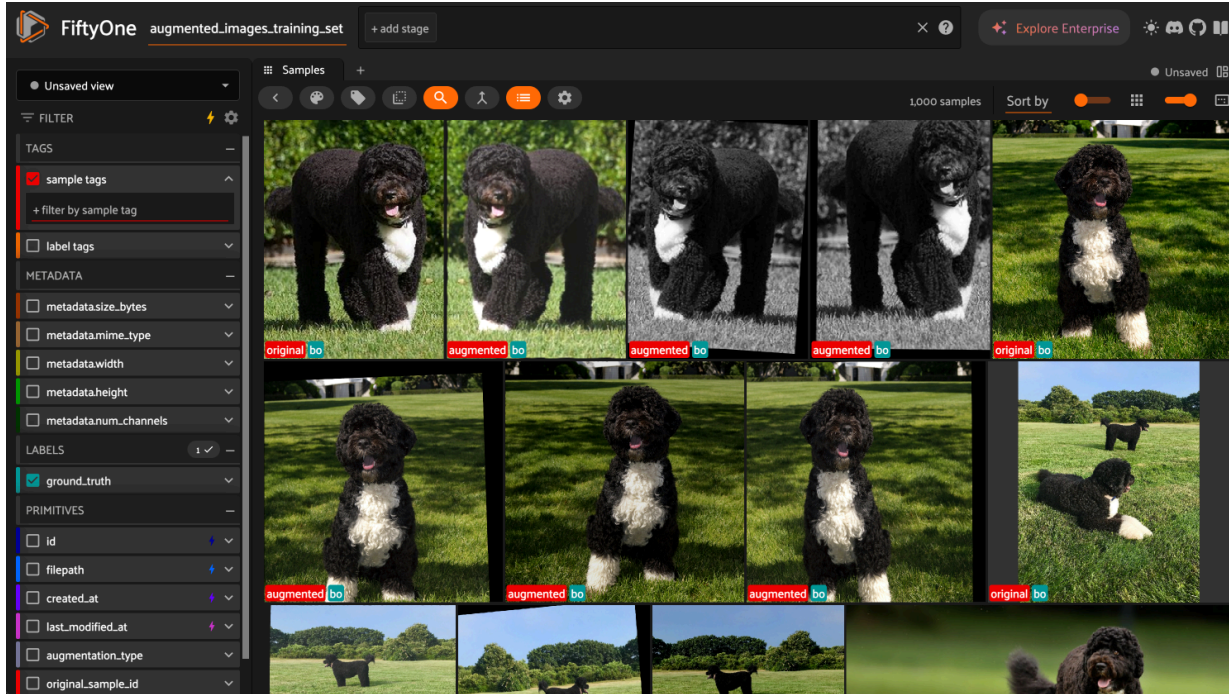
I strongly encourage you to:

1. **Train without Imagenet-1K weights:** Observe the baseline performance of a VGG16 model trained with random weights
2. **Add weighted loss:** Compare the results and note improvements
3. **Experiment with different weight values for the minority class on the loss functions:** Try manual adjustments to understand sensitivity
4. **Analyze confusion matrices:** Use FiftyOne to visualize the classification results for both approaches

This hands-on experimentation will deepen your understanding of how loss weighting affects model behavior and performance. The "train -> analyze -> improve" loop should be one the takeaways from this tutorial.

Data Augmentation for Robust Training

Data augmentation increases training data diversity and improves model generalization. This is intuitive, but seeing is believing. Before feeding augmented images into our model, it's wise to inspect them. Do they still look like valid images? FiftyOne makes this easy. After defining our `train_transforms`, we can apply them to samples and visualize the result directly in the app, confirming that our augmentations are behaving as expected.



Original vs Augmented Images

```
train_transforms = transforms.Compose([

    # Take a random crop of size 224 in the image
    transforms.RandomResizedCrop(224),

    # A dog is still a dog if we flip it left <-> right
    transforms.RandomHorizontalFlip(),

    # Random changes in color or contrast
    transforms.ColorJitter(brightness=0.2, contrast=0.2),

    # Convert to torch Image tensor
    transforms.ToImage(),

    # Standard transformations for Imagenet-pretrained models
    transforms.ToDtype(torch.float32, scale=True),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225])

])
```

Why These Transformations Work:

Data Augmentation Principle: The first three transforms create variations of your training data without changing the fundamental content. Think of it like teaching the model through multiple photos of the same object from different angles and lighting. The model learns to recognize the object regardless of these variations.

Normalization Principle: The ImageNet-1k statistics (those specific mean and standard deviation values) represent the "typical" color distribution across a large dataset with a wide variety of images. By normalizing to these values, we're speaking the same "language" that pre-trained models were taught, making transfer learning much more effective. The red, green, and blue intensities in our data become interpretable to the model as similar to the intensities in the original ImageNet-1k dataset.

The transformations follow a logical order: spatial augmentations → color augmentations → tensor conversion → normalization. This pipeline is a proven pattern for computer vision training.

These transformations create variations of training images while preserving semantic content, helping the model generalize to new examples. A very neat aspect of FiftyOne is that we can visualize the augmentations that we apply to our samples before sending them to the model. [Training set augmentation is a form of dataset curation](#) and it's sensible to spend effort on it. Some augmentations will be more successful than others. Experiment with this!

Model Evaluation and Analysis

After training, we evaluate model performance using FiftyOne's evaluation tools:

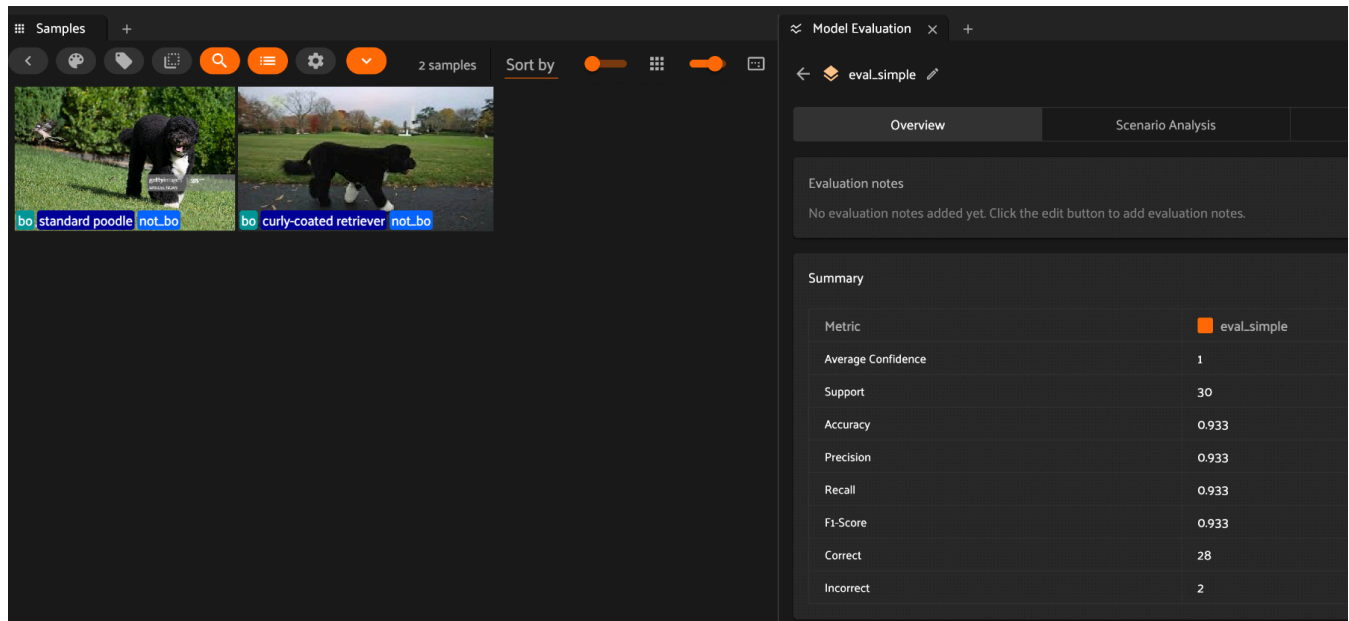
```
# Apply trained model to test set
test_view = dataset.match_tags("test")
test_view.apply_model(trained_model, label_field="predictions")

# Evaluate predictions
results = test_view.evaluate_classifications(
    "predictions",
    gt_field="ground_truth",
    eval_key="eval_simple"
)

print(f"Precision: {results.precision():.3f}")
print(f"Recall: {results.recall():.3f}")
```

Analyzing Mistakes and Edge Cases

FiftyOne allows us to explore model failures and identify patterns in misclassified examples:



Error Analysis

```
# Find all samples where the prediction does not match the ground
truth

mistakes = test_view.match(
    F("predictions.label") != F("ground_truth.label")
)

# Sort by confidence to find the most confident mistakes

mistakes = mistakes.sort_by("predictions.confidence", reverse=True)
```

This analysis reveals edge cases and potential areas for improvement, such as: images with atypical aspect ratios and text captions on the body of the subject. These mistakes usually appear in the minority class, and give us hints on what to try next.

By sorting our mistakes in FiftyOne, we've uncovered our model's weak spots: images with unusual aspect ratios and embedded text. This analysis is not the end of the project. It's the start of the next iteration. Based on these findings, our next steps would be to:

1. Use FiftyOne to tag these failure cases.
2. Either find more training images with similar characteristics (e.g., photos of dogs with wide aspect ratios)
3. Adjust our data augmentation strategy to include more aggressive resizing and perhaps even an augmentation that randomly places text on images. This data-centric

loop of train -> analyze -> improve is the essence of building powerful real-world models.

4. Use a loss function that is aware of our class imbalance.

This analysis of mistakes is not the end of the project, but rather the beginning of the next iteration. By understanding why our model fails, we can go back and improve our dataset or training strategy. This is the core of the data-centric AI workflow.

Conclusion: A Blueprint for Real-World Transfer Learning

Successful transfer learning requires more than adapting a model's architecture; it demands a rigorous, data-centric workflow. Using FiftyOne, we transformed a general-purpose ImageNet model into a specialized "Bo or Not Bo" classifier, navigating common challenges in the process.

Some takeaways:

1. **Always Start with Your Data:** Use tools like FiftyOne to visualize and understand your dataset's characteristics before you start training.
2. **Cleanliness is Key:** Focus on identifying and resolving data quality issues like duplicates and leakage to ensure reliable model evaluation.
3. **Choose the Right Tools for the Problem:** Don't use a one-size-fits-all approach. Adapt your tools, like your loss function, to the specific challenges of your dataset.
4. **Analyze, Don't Just Measure:** Go beyond accuracy or F1 scores. Use your model's failures as a guide to iterate and improve your data and your model.